Import, joins, and missingness with dplyr

Workflow labs use the variant template: Goal → Approach → Execution → Check → Report.

Learning objectives

- Use `filter()`, `select()`, `mutate()`, `arrange()`, `group_by()`, and `summarise()` to carry out routine data management with palmerpenguins.
- Join two tables on a shared key and explain the difference between inner, left, and anti joins.
- Spot, quantify, and respond to missing data without letting NAs propagate silently into a summary.

Prerequisites

Labs 1.2 and 1.3.

Background

The bulk of analytic time is spent moving data from the shape it arrived in to the shape an analysis requires. dplyr names the five verbs you need for most of that work — filter rows, select columns, mutate to create new variables, arrange to sort, summarise with an optional grouping. If you learn these six verbs well you will write less code and hide fewer bugs.

Joins are the other half of data management. Real datasets arrive in pieces — a demographics table, a laboratory table, a follow-up table — that must be connected on a shared identifier. `left_join()` keeps all rows on the left side; `inner_join()` keeps only matches; `anti_join()` returns the rows with no match, which is often where data-quality problems live.

Missingness is not rare. A realistic clinical dataset has NAs in at least half its variables, and the NAs are almost never uniformly distributed. The first question to ask of any new dataset is *how much is missing, and where?* The second is *why?* Missing-completely-at- random, missing-at-random, and missing-not-at-random are three distinct problems with different consequences for the analysis.

Setup

```
library(tidyverse)
library(palmerpenguins)
set.seed(42)
theme_set(theme_minimal(base_size = 12))
```

1. Goal

Reshape the `penguins` dataset with the standard dplyr verbs, join a derived table back on, and describe the pattern of missingness.

2. Approach

```r
penguins |>
  filter(!is.na(body_mass_g)) |>
  select(species, sex, bill_length_mm, body_mass_g) |>
  mutate(mass_kg = body_mass_g / 1000) |>
  arrange(desc(mass_kg)) |>
  head(5)
```

```r
species_summary <- penguins |>
  group_by(species, sex) |>
  summarise(
    n    = n(),
    mean_mass = mean(body_mass_g, na.rm = TRUE),
    sd_mass   = sd(body_mass_g,   na.rm = TRUE),
    .groups = "drop"
  )
species_summary
```

3. Execution

Simulate a small "islands" table with information not in `penguins`, and join it back.

```r
  island = c("Biscoe", "Dream", "Torgersen"),
  lat    = c(-65.43, -64.73, -64.77),
  researcher = c("Anna", "Ben", "Carla")
)

penguins_aug <- penguins |>
  left_join(islands, by = "island")

penguins_aug |>
  select(species, island, researcher, lat) |>
  head(5)
```

```r
penguins |>
  anti_join(islands, by = "island")
```

```r
inner_join(penguins, islands, by = "island") |>
  nrow()
```

4. Check

Quantify missingness at a glance.

```r
  summarise(across(everything(), ~ mean(is.na(.)))) |>
  pivot_longer(everything(),
               names_to = "variable",
               values_to = "frac_missing") |>
  arrange(desc(frac_missing))
missingness
```

```r
missingness |>
  ggplot(aes(frac_missing, reorder(variable, frac_missing))) +
  geom_col(fill = "grey60") +
  labs(x = "Fraction missing", y = NULL)
```

Now inject missingness to show how NAs move through a pipeline.

```r
inj <- penguins |>
  mutate(body_mass_g = if_else(runif(n) < 0.1, NA_real_, body_mass_g))
mean(inj$body_mass_g)            # NA - propagates
mean(inj$body_mass_g, na.rm = TRUE)
```

5. Report

> Of the 344 rows in `palmerpenguins::penguins`, sex was missing in `sum(is.na(penguins$sex))` rows (`round(100 * mean(is.na(penguins$sex)), 1)`%) and the four morphometric variables were missing in two rows apiece. After an inner join with a three-row `islands` table, all rows matched. Missingness was quantified per-variable and any summary that took a mean used `na.rm = TRUE` explicitly; silent NA propagation would otherwise have produced an NA mean.

Always look at the pattern of missingness before running an analysis. If NAs cluster in one group, a list-wise delete changes the comparison you thought you were making.

Common pitfalls

- Forgetting `na.rm = TRUE` in a `mean()` and getting back an NA.
- Confusing inner and left join and silently dropping rows.
- Using row numbers as join keys after a reshape (they will not match).
- Imputing missingness by hand and forgetting to flag the imputed rows.

Further reading

- Wickham H, Grolemund G. *R for Data Science*, chapter on data transformation.
- Little RJA, Rubin DB. *Statistical Analysis with Missing Data*.

Session info

```r
```

See also — chapter index

- Methods in this chapter
- Find your method (Chapter 0)